Práctica 4: Pruebas unitarias

1. Tabla de casos

Casos normales

Casos	Entrada	Resultado Esperado	Regla	Justificación
1	10 min, normal, boleto no perdido	$0.00	R5	10 min está dentro de "1 a 15": no se cobra
2	45 min, normal, boleto no perdido	$20.00	R6	45 min está a mitad de "16 a 60": se cobra la tarifa de la primera hora
3	90 min, normal, boleto no perdido	$35.00	R7	$20 de la primera hora más $15 de una hora adicional iniciada

Casos frontera

Casos	Entrada	Resultado Esperado	Regla	Justificación
4	0 min, normal, boleto no perdido	$0.00	R4	0 minutos: gratis
5	1 min, normal, boleto no perdido	$0.00	R5	Primer valor del rango "1 a 15"
6	14 min, normal, boleto no perdido	$0.00	R5	Justo antes del límite de 15
7	15 min, normal, boleto no perdido	$0.00	R5	Último minuto gratis ("de 1 a 15")
8	16 min, normal, boleto no perdido	$20.00	R6	Primer minuto que ya se cobra: cambia de $0 a $20
9	17 min, normal, boleto no perdido	$20.00	R6	Justo después del límite
10	60 min, normal, boleto no perdido	$20.00	R6	Último minuto del rango "16 a 60"
11	120 min, normal, boleto no perdido	$35.00	R7	Exactamente 1 hora adicional (60 + 60)
12	121 min, normal, boleto no perdido	$50.00	R7	Primera hora + 2 horas adicionales iniciadas

Entradas inválidas

Casos	Entrada	Resultado Esperado	Regla	Justificación
13	-1 min, normal, boleto no perdido	ERROR (ValueError)	R1 y R10	Minutos negativos
14	30 min, Normal, boleto no perdido	ERROR (ValueError)	R2 y R11	Los valores válidos son exactamente "normal" y "frecuente" (distingue mayúsculas)

Interacción entre reglas

Casos	Entrada	Resultado Esperado	Reglas	Justificación
15	61 min, frecuente, boleto no perdido	$31.50	R7 y R8	Más de 60 min con cliente frecuente: $35.00 menos el 10 %
16	15 min, frecuente, boleto no perdido	$0.00	R5 y R8	Frontera de 15 min combinada con descuento: 0 menos 10 % sigue siendo 0
17	10 min, frecuente, boleto perdido	$300.00	R3 y R8	Boleto perdido: el cobro es $300.00 sin importar el tipo de cliente, el descuento de frecuente no aplica

Números decimales

Casos	Entrada	Resultado Esperado	Reglas	Justificación
18	181 min, frecuente, boleto no perdido	$58.50	R7 y R8	$65.00 menos el 10 % da un resultado con decimales; se compara con tolerancia (pytest.approx)

Caso adicional

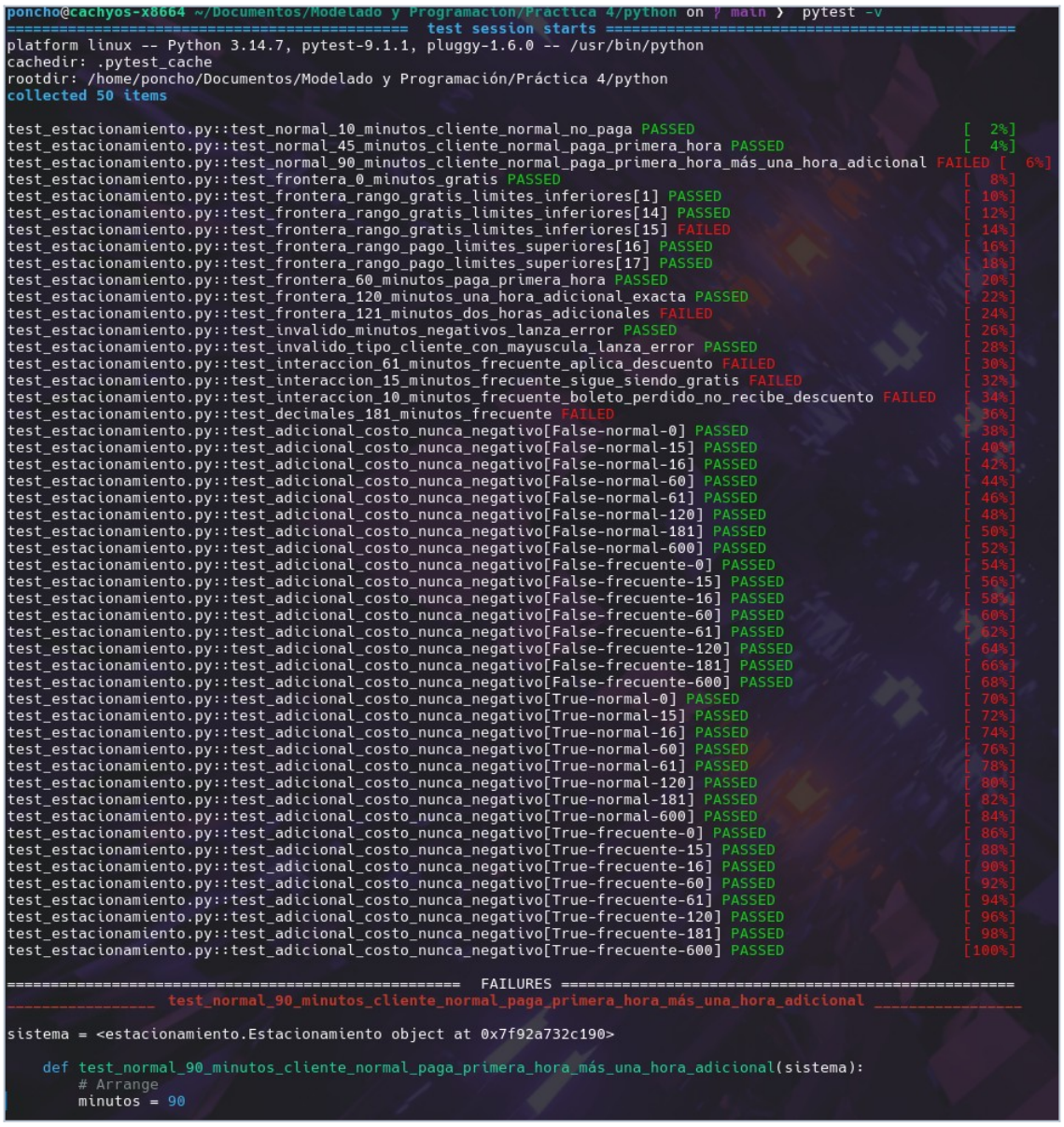
Casos	Entrada	Resultado Esperado	Reglas	Justificación
19	Cualquier valor de 0 a 600 min, ambos tipos, con y sin boleto perdido	Resultado mayor o igual a $0.00	R9	El costo nunca puede ser negativo

2. Reglas cubiertas

Regla	Descripción	Casos que la cubren
R1	Minutos enteros ≥ 0	13
R2	Tipos válidos: "normal", "frecuente"	14
R3	Boleto perdido → $300.00	17
R4	0 min → $0.00	4
R5	1–15 min → $0.00	1, 5, 6, 7, 16
R6	16–60 min → $20.00	2, 8, 9, 10
R7	>60 min → $20 + $15 por hora iniciada	3, 11, 12, 15, 18
R8	Frecuente: −10 % (excepto boleto perdido)	15, 16, 17, 18
R9	Costo nunca negativo	19
R10	Minutos negativos → error	13
R11	Tipo inválido → error	14

3. Evidencia de ejecución

3.1 Ejecución con la implementación original (7 fallos)



Captura 1 de 3: resultado de pytest -v y comienzo del reporte de fallos.



Captura 2 de 3: continuación del reporte de fallos.



Captura 3 de 3: fin del reporte de fallos y resumen (7 failed, 43 passed).

3.2 Análisis de los fallos

La suite detectó 7 fallos que corresponden a 3 bugs independientes en la implementación:

Bug 1

- **Prueba que lo detecta:** `test_frontera_rango_gratis_limites_inferiores[15]`
- **Esperado:** $0.00 (R5: "De 1 a 15 minutos, el costo es $0.00")
- **Obtenido:** $20.00
- **Causa:** la implementación usa `elif minutos < 15:` en lugar de `elif minutos <= 15:`. El minuto 15 debería caer en el rango gratis, pero cae en R6.
- **Corrección:** cambiar `<` por `<=`.

Bug 2

- **Pruebas que lo detectan:** 90 min normal, 121 min normal, 61 min frecuente y 181 min frecuente.
- **Causa:** la implementación usa `(minutos - 60) // 60` (división entera), lo cual redondea hacia abajo las horas adicionales. El modelo (R7) exige contar cada hora adicional iniciada, es decir, redondear hacia arriba.
- **Corrección:** cambiar

```
horas_adicionales = (minutos - 60) // 60
```

por

```
minutos_restantes = minutos - 60
horas_adicionales = (minutos_restantes + 59) // 60
```

Bug 3

- **Prueba que lo detecta:** `test_interaccion_10_minutos_frecuente_boleto_perdido_no_recibe_descuento`
- **Esperado:** $300.00 (R3: el boleto perdido cuesta $300.00 sin importar el tipo de cliente; R8: el descuento no aplica cuando se perdió el boleto)
- **Obtenido:** $270.00
- **Causa:** la implementación ejecuta `if tipo_cliente == "frecuente":` sin revisar `boleto_perdido`, así que multiplica por 0.90 también los $300.00.
- **Corrección:** agregar la condición `and not boleto_perdido`.

Tabla de fallos

#	Prueba	Esperado	Obtenido	Bug
1	90 min normal	$35.00	$20.00	Bug 2 (R7)
2	15 min normal	$0.00	$20.00	Bug 1 (R5)
3	121 min normal	$50.00	$35.00	Bug 2 (R7)
4	61 min frecuente	$31.50	$18.00	Bug 2 (R7)
5	15 min frecuente	$0.00	$18.00	Bug 1 (R5)
6	10 min frecuente, boleto perdido	$300.00	$270.00	Bug 3 (R3 y R8)
7	181 min frecuente	$58.50	$45.00	Bug 2 (R7)

3.3 Ejecución después de corregir los tres bugs

```
poncho@cachyos-x8664 ~/Documentos/Modelado y Programacion/Practica 4/python on ᚠ main ❯  pytest -v
============================================  test session starts ============================================
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python
cachedir: .pytest_cache
rootdir: /home/poncho/Documentos/Modelado y Programación/Práctica 4/python
collected 49 items

test_estacionamiento.py::test_normal_10_minutos_cliente_normal_no_paga PASSED                                [  2%]
test_estacionamiento.py::test_normal_45_minutos_cliente_normal_paga_primera_hora PASSED                      [  4%]
test_estacionamiento.py::test_normal_90_minutos_cliente_normal_paga_primera_hora_más_una_hora_adicional PASSED [  6%
test_estacionamiento.py::test_frontera_0_minutos_gratis PASSED                                               [  8%]
test_estacionamiento.py::test_frontera_rango_gratis_limites_inferiores[1] PASSED                             [ 10%]
test_estacionamiento.py::test_frontera_rango_gratis_limites_inferiores[14] PASSED                            [ 12%]
test_estacionamiento.py::test_frontera_rango_gratis_limites_inferiores[15] PASSED                            [ 14%]
test_estacionamiento.py::test_frontera_rango_pago_limites_superiores[16] PASSED                              [ 16%]
test_estacionamiento.py::test_frontera_rango_pago_limites_superiores[17] PASSED                              [ 18%]
test_estacionamiento.py::test_frontera_60_minutos_paga_primera_hora PASSED                                   [ 20%]
test_estacionamiento.py::test_frontera_120_minutos_una_hora_adicional_exacta PASSED                          [ 22%]
test_estacionamiento.py::test_frontera_121_minutos_dos_horas_adicionales PASSED                              [ 24%]
test_estacionamiento.py::test_invalido_minutos_negativos_lanza_error PASSED                                  [ 26%]
test_estacionamiento.py::test_invalido_tipo_cliente_con_mayuscula_lanza_error PASSED                         [ 28%]
test_estacionamiento.py::test_interaccion_61_minutos_frecuente_aplica_descuento PASSED                       [ 30%]
test_estacionamiento.py::test_interaccion_15_minutos_frecuente_sigue_siendo_gratis PASSED                    [ 32%]
test_estacionamiento.py::test_decimales_181_minutos_frecuente PASSED                                         [ 34%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-0] PASSED                          [ 36%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-15] PASSED                         [ 38%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-16] PASSED                         [ 40%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-60] PASSED                         [ 42%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-61] PASSED                         [ 44%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-120] PASSED                        [ 46%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-181] PASSED                        [ 48%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-normal-600] PASSED                        [ 51%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-0] PASSED                       [ 53%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-15] PASSED                      [ 55%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-16] PASSED                      [ 57%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-60] PASSED                      [ 59%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-61] PASSED                      [ 61%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-120] PASSED                     [ 63%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-181] PASSED                     [ 65%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[False-frecuente-600] PASSED                     [ 67%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-0] PASSED                           [ 69%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-15] PASSED                          [ 71%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-16] PASSED                          [ 73%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-60] PASSED                          [ 75%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-61] PASSED                          [ 77%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-120] PASSED                         [ 79%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-181] PASSED                         [ 81%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-normal-600] PASSED                         [ 83%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-0] PASSED                        [ 85%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-15] PASSED                       [ 87%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-16] PASSED                       [ 89%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-60] PASSED                       [ 91%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-61] PASSED                       [ 93%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-120] PASSED                      [ 95%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-181] PASSED                      [ 97%]
test_estacionamiento.py::test_adicional_costo_nunca_negativo[True-frecuente-600] PASSED                      [100%]

============================================  49 passed in 0.04s =============================================
```

4. Respuestas de análisis

¿Qué diferencia hay entre un caso normal y uno frontera?

Un caso normal usa valores intermedios dentro de un rango de comportamiento, lejos de los límites. Un caso frontera usa valores exactamente en el límite donde el comportamiento cambia.

¿Cuál de sus pruebas consideran más importante y por qué?

La prueba de 90 minutos (R7) es la más importante porque detectó un bug en la implementación: el código devolvía $20.00 cuando el modelo exige $35.00. Si no hubiéramos probado valores mayores a 60 min, el bug habría pasado desapercibido.

¿Encontro algún comportamiento de la implementación que no coincida con el modelo?

Sí, encontramos dos bugs:

- Bug 1 (R5): para 15 minutos exactos, la implementación cobra $20.00 cuando el modelo dice $0.00. La causa es que usa minutos < 15 en lugar de minutos <= 15.
- Bug 2 (R7): para más de 60 minutos, la implementación redondea hacia abajo las horas adicionales por ejemplo:
  - 61 min normal: esperado $35.00, obtenido $20.00
  - 90 min normal: esperado $35.00, obtenido $20.00
  - 121 min normal: esperado $50.00, obtenido $35.00

La causa es que usa (minutos - 60) // 60 en lugar de redondear hacia arriba.

¿Una suite con 100% de pruebas demuestra que el programa es correcto? Explique

No. Una suite al 100% solo demuestra que el programa pasó los casos que decidimos comprobar, no que sea correcto en todos los escenarios posibles.

De hecho, en esta práctica la implementación original falla 6 casos. Si solo hubiéramos probado valores "cómodos" (10, 30, 45 min con cliente normal), la suite habría dado 100 % passed y los bugs de R5 y R7 habrían pasado desapercibidos.

Si una IA genera 50 pruebas ¿qué tendría que revisar una persona antes de confiar en ellas?

Antes de confiar en pruebas generadas por IA, una persona debe revisar:

1. Que los resultados esperados provengan del modelo, no de la implementación (evitar que la IA copie el comportamiento del código).
2. Que cubran fronteras (15/16, 60/61, 120/121).
3. Que incluyan entradas inválidas y verifiquen excepciones.
4. Que incluyan interacciones entre reglas (frecuente + boleto perdido, frecuente + >60 min).
5. Que usen tolerancia decimal donde corresponda.
6. Que no dupliquen casos sin aportar algo nuevo.
7. Que no inventen reglas que no están en el modelo.